

Presentazione del progetto Audire

Piattaforma centralizzata per la gestione dei Casting

Gennaro Carmine Tozza Gabriele Malanga

Università degli Studi di Salerno
Corso di Ingegneria del Software

Gennaio 2026



- 1 Introduzione
- 2 Requirements Analysis Document (RAD)
- 3 System Design Document (SDD)
- 4 Object Design (ODD)
- 5 Testing

Il processo di casting oggi è caratterizzato da significative inefficienze:

- **Comunicazione frammentata:** email, fogli di calcolo, messaggi sparsi

Il processo di casting oggi è caratterizzato da significative inefficienze:

- **Comunicazione frammentata:** email, fogli di calcolo, messaggi sparsi
- **Mancanza di tracciabilità:** candidature perse tra cartelle disorganizzate

Il processo di casting oggi è caratterizzato da significative inefficienze:

- **Comunicazione frammentata:** email, fogli di calcolo, messaggi sparsi
- **Mancanza di tracciabilità:** candidature perse tra cartelle disorganizzate
- **Processo non standardizzato:** ogni produzione segue procedure diverse

Il processo di casting oggi è caratterizzato da significative inefficienze:

- **Comunicazione frammentata:** email, fogli di calcolo, messaggi sparsi
- **Mancanza di tracciabilità:** candidature perse tra cartelle disorganizzate
- **Processo non standardizzato:** ogni produzione segue procedure diverse
- **Valutazione disorganizzata:** feedback sparsi e difficili da confrontare

Negli ultimi 5 anni il settore dell'intrattenimento ha vissuto cambiamenti significativi:

Boom delle Piattaforme Streaming

Netflix, Amazon Prime, Disney+ hanno aumentato del **300%** la produzione di contenuti originali dal 2020

Internazionalizzazione delle Produzioni

Le produzioni non sono più limitate geograficamente: serie TV prodotte in Italia cercano attori in tutta Europa

Audire (dal latino "ascoltare") è una piattaforma web che centralizza e digitalizza l'intero processo di casting.

Obiettivi Principali

- 1 Abbinare talenti e opportunità in modo efficiente
- 2 Gestire le candidature in modo strutturato
- 3 Garantire tracciabilità completa del processo
- 4 Facilitare la comunicazione tra stakeholder

Il sistema coinvolge tre categorie principali di utenti:

-  **Performer** Artisti che cercano e si candidano alle audizioni
-  **Casting Director (CD)** Responsabili della selezione e valutazione
-  **Production Manager (PM)** Supervisor che creano le produzioni e gestiscono i team

Dopo aver definito il Problem Statement si è passato alla fase di **analisi dei requisiti**.

L'output di questa fase è il **RAD** (Requirements Analysis Document), questo documento:

- Descrive completamente il sistema in termini di requisiti funzionali e non funzionali
- Funge da base contrattuale tra il cliente e gli sviluppatori

I requisiti funzionali descrivono la funzionalità di alto livello del sistema.

RF1 - Gestione Account

Registrazione, login, gestione profilo con informazioni specifiche per ruolo

RF2 - Gestione Produzioni

I PM creano, visualizzano e modificano produzioni, assegnano Casting Director, monitorano l'avanzamento

RF3 - Gestione Casting

I CD creano, modificano e pubblicano annunci dettagliati con requisiti e scadenze con la possibilità di visualizzare le candidature inviate dai performer

RF4 - Sistema di Candidatura

I Performer cercano casting, si candidano, monitorano lo status

RF5 - Valutazione Candidature

I CD valutano candidati, aggiornano status, forniscono feedback

I requisiti non funzionali descrivono requisiti a livello utente che non sono direttamente correlati alla funzionalità.

Performance Tempi di caricamento < 3 secondi, ricerche < 2 secondi

Usabilità Interfaccia intuitiva, responsive design, validazione real-time

Sicurezza Password con hashing sicuro, protezione SQL injection

Conformità GDPR compliance, privacy policy chiara

Implementazione Database relazionale, architettura three-tier

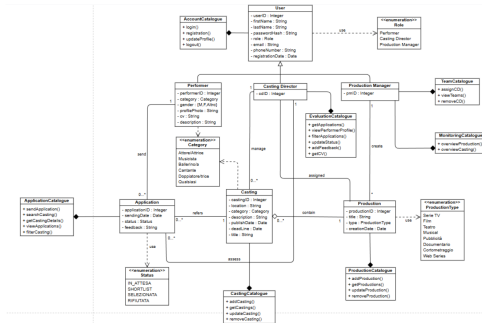
Il Requirements Analysis Document non specifica solo i requisiti funzionali e non funzionali ma anche i modelli di sistema.

Quest'ultimi descrivono:

- Gli scenari e i casi d'uso
- Il modello a oggetti
- I modelli dinamici del sistema
- L'interfaccia utente e i percorsi di navigazione

12/72

◀ ◻ ▶ ◀ ▢ ▶ ◀ ≡ ▶ ◀ ≡≡ ▶ ≡≡≡ ↺ 🔍



Per illustrare la coerenza della progettazione, analizziamo l'evoluzione di uno scenario:

Un Performer si candida ad un Casting aperto

Attore: Maria (Performer)

Obiettivo: Trovare un ruolo per la sua carriera.

- **Accesso:** Maria effettua il login e accede alla Dashboard

Attore: Maria (Performer)

Obiettivo: Trovare un ruolo per la sua carriera.

- **Accesso:** Maria effettua il login e accede alla Dashboard
- **Scoperta:** Nota l'annuncio per "*LifeInUnisa*" (creato dal CD Laura)

Attore: Maria (Performer)

Obiettivo: Trovare un ruolo per la sua carriera.

- **Accesso:** Maria effettua il login e accede alla Dashboard
- **Scoperta:** Nota l'annuncio per "*LifeInUnisa*" (creato dal CD Laura)
- **Analisi:** Clicca sull'annuncio e visualizza i dettagli:
 - Descrizione ruolo ("Protagonista Femminile")
 - Date riprese e Location
 - Requisiti fisici e skill richieste

Attore: Maria (Performer)

Obiettivo: Trovare un ruolo per la sua carriera.

- **Accesso:** Maria effettua il login e accede alla Dashboard
- **Scoperta:** Nota l'annuncio per "*LifeInUnisa*" (creato dal CD Laura)
- **Analisi:** Clicca sull'annuncio e visualizza i dettagli:
 - Descrizione ruolo ("Protagonista Femminile")
 - Date riprese e Location
 - Requisiti fisici e skill richieste
- **Decisione:** Il ruolo corrisponde al suo profilo → Decide di candidarsi

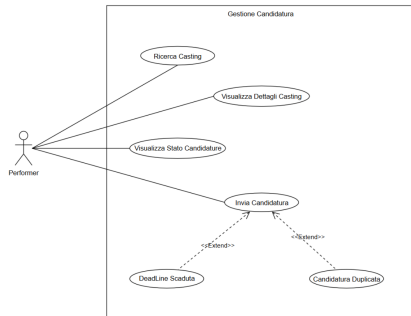
- 1 **Azione:** Click sul pulsante "Candidati"

- ① **Azione:** Click sul pulsante "**Candidati**"
- ② **Verifica:** Il sistema mostra il riepilogo dei dati che verranno inviati:
 - CV e Foto Profilo (già presenti nel sistema)
 - Informazioni professionali

- ① **Azione:** Click sul pulsante "**Candidati**"
- ② **Verifica:** Il sistema mostra il riepilogo dei dati che verranno inviati:
 - CV e Foto Profilo (già presenti nel sistema)
 - Informazioni professionali
- ③ **Conferma:** Maria conferma l'invio

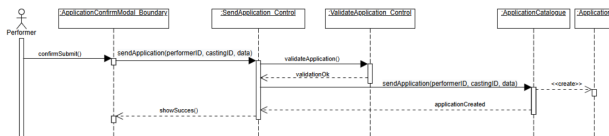
- 1 **Azione:** Click sul pulsante "**Candidati**"
- 2 **Verifica:** Il sistema mostra il riepilogo dei dati che verranno inviati:
 - CV e Foto Profilo (già presenti nel sistema)
 - Informazioni professionali
- 3 **Conferma:** Maria conferma l'invio
- 4 **Risultato:**
 - Il sistema registra la candidatura nel DB
 - Lo status è impostato su "In Attesa"
 - Maria viene reindirizzata alla sezione "*Le Mie Candidature*"

L'interazione tra l'attore Performer e il Sistema

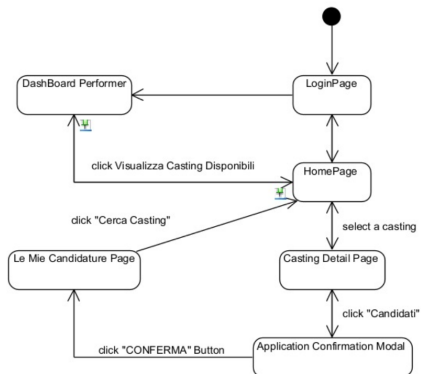


L'attore *Performer*, previa autenticazione, accede alla lista casting e avvia la procedura di candidatura.

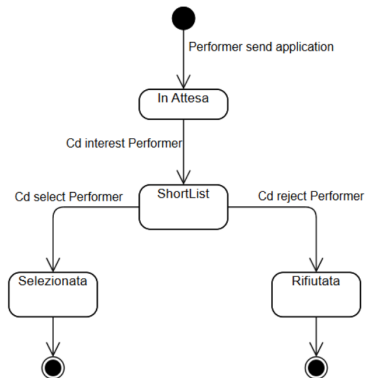
Il flusso temporale della candidatura



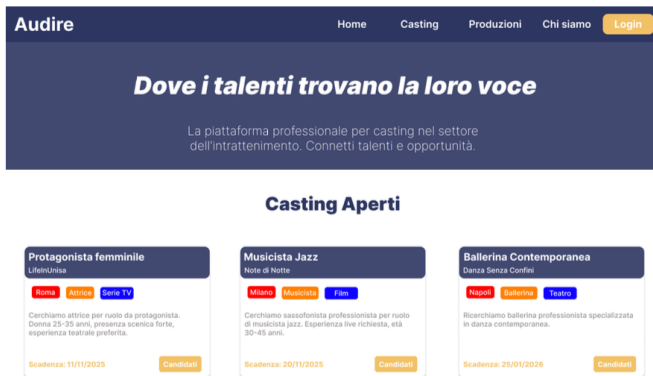
1. Click "Candidati" → 2. Verifica Requisiti → 3. Salvataggio Candidatura → 4. Notifica successo



Il seguente state diagram illustra il ciclo di vita di una candidatura (Application) nel sistema Audire.



I mock-up rappresentano la produzione completa dell'interfaccia utente, definiscono con completezza e senza ambiguità i requisiti.



Mock-up della HomePage di Audire

Successivamente alla fase di analisi dei requisiti, si passa al System Design. Quest'ultimo descrive:

- Gli obiettivi di progettazione stabiliti dal progetto

Successivamente alla fase di analisi dei requisiti, si passa al System Design. Quest'ultimo descrive:

- Gli obiettivi di progettazione stabiliti dal progetto
- La decomposizione in sottosistemi

Successivamente alla fase di analisi dei requisiti, si passa al System Design. Quest'ultimo descrive:

- Gli obiettivi di progettazione stabiliti dal progetto
- La decomposizione in sottosistemi
- La mappatura hardware/software

Successivamente alla fase di analisi dei requisiti, si passa al System Design. Quest'ultimo descrive:

- Gli obiettivi di progettazione stabiliti dal progetto
- La decomposizione in sottosistemi
- La mappatura hardware/software
- La gestione dei dati

Successivamente alla fase di analisi dei requisiti, si passa al System Design. Quest'ultimo descrive:

- Gli obiettivi di progettazione stabiliti dal progetto
- La decomposizione in sottosistemi
- La mappatura hardware/software
- La gestione dei dati
- Il controllo degli accessi e le condizioni al contorno

Gli obiettivi primari che guidano l'architettura di Audire sono:

DG_1 - Criteri di Performance

- **Tempi di risposta:** Caricamento pagine in $< 3s$ e risultati ricerca in $< 2s$
- **Upload File:** Supporto fino a 10 MB con feedback visivo
- **Dati Persistenti:** Utilizzo di DB relazionale (MySQL) per accesso rapido

DG_2 - Criteri di Affidabilità

- **Robustezza:** Gestione controllata degli errori e validazione form real-time
- **Affidabilità:** Transazionalità delle operazioni critiche
- **Sicurezza:** Autenticazione sicura, HTTPS, RBAC e hashing password

DG_3 - Criteri di Costo

Valutazione preventiva dei costi di sviluppo, manutenzione e amministrazione

DG_4 - Criteri di Mantenimento

- **Modificabilità:** Possibilità di alterare funzionalità esistenti senza modificare l'intera struttura

DG_5 - Criteri di Utente

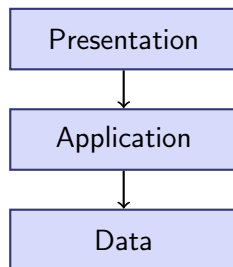
- **Utilità:** Soddisfacimento dei requisiti funzionali per fornire valore reale
- **Usabilità:** Interfaccia intuitiva e Responsive Design (desktop, tablet, smartphone)

Trade-off	Descrizione
Performance vs Sicurezza	Si è scelto di privilegiare la sicurezza dei dati degli utenti rispetto a miglioramenti marginali delle performance. <i>Conseguenza:</i> I tempi di risposta potrebbero risultare di poco superiori ai 3 secondi per garantire la crittografia e i controlli di accesso.

Stile: Ibrido Client-Server e MVC
(Model-View-Controller)

Struttura *Three-Layer* per ogni sottosistema:

- 1 **Presentation Layer:** Interfaccia grafica (GUI)
- 2 **Application Layer:** Logica di elaborazione
- 3 **Data Layer:** Accesso ai dati persistenti



Il sistema è suddiviso nei seguenti sottosistemi principali:

- **Gestione Account:** Registrazione, login e gestione profili

Il sistema è suddiviso nei seguenti sottosistemi principali:

- **Gestione Account:** Registrazione, login e gestione profili
- **Gestione Produzione:** CRUD delle produzioni (PM)

Il sistema è suddiviso nei seguenti sottosistemi principali:

- **Gestione Account:** Registrazione, login e gestione profili
- **Gestione Produzione:** CRUD delle produzioni (PM)
- **Gestione Team:** Assegnazione dei Casting Director (PM)

Il sistema è suddiviso nei seguenti sottosistemi principali:

- **Gestione Account:** Registrazione, login e gestione profili
- **Gestione Produzione:** CRUD delle produzioni (PM)
- **Gestione Team:** Assegnazione dei Casting Director (PM)
- **Gestione Casting:** Creazione annunci (CD)

Il sistema è suddiviso nei seguenti sottosistemi principali:

- **Gestione Account:** Registrazione, login e gestione profili
- **Gestione Produzione:** CRUD delle produzioni (PM)
- **Gestione Team:** Assegnazione dei Casting Director (PM)
- **Gestione Casting:** Creazione annunci (CD)
- **Gestione Candidature:** Invio candidature (Performer)

Il sistema è suddiviso nei seguenti sottosistemi principali:

- **Gestione Account:** Registrazione, login e gestione profili
- **Gestione Produzione:** CRUD delle produzioni (PM)
- **Gestione Team:** Assegnazione dei Casting Director (PM)
- **Gestione Casting:** Creazione annunci (CD)
- **Gestione Candidature:** Invio candidature (Performer)
- **Gestione Valutazione:** Selezione e feedback (CD)

Il sistema è suddiviso nei seguenti sottosistemi principali:

- **Gestione Account:** Registrazione, login e gestione profili
- **Gestione Produzione:** CRUD delle produzioni (PM)
- **Gestione Team:** Assegnazione dei Casting Director (PM)
- **Gestione Casting:** Creazione annunci (CD)
- **Gestione Candidature:** Invio candidature (Performer)
- **Gestione Valutazione:** Selezione e feedback (CD)
- **Monitoraggio:** Statistiche aggregate (PM)

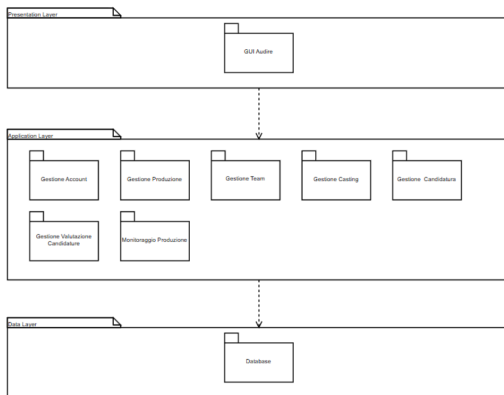


Figura: Macro-decomposizione del sistema

Questo sottosistema gestisce la ricerca, visualizzazione e invio delle candidature da parte dei Performer.

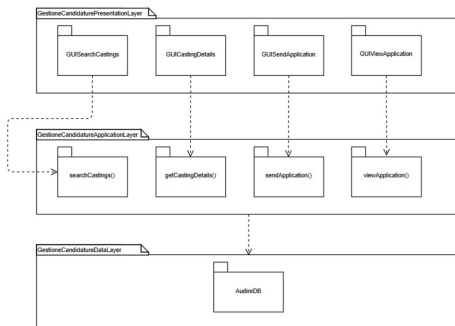


Figura: Diagramma delle classi di analisi (Gestione Candidature)

Sottosistema	Descrizione
Gestione Candidature	Gestisce la ricerca dei casting e l'invio delle candidature da parte dei Performer.
Servizio	Descrizione
searchCastings()	Ricerca casting aperti con filtri per location, categoria e tipologia
getCastingDetails()	Visualizza i dettagli completi di un casting
sendApplication()	Invia la candidatura a un casting
viewApplications()	Visualizza lo storico delle candidature con status e feedback

Deployment Diagram

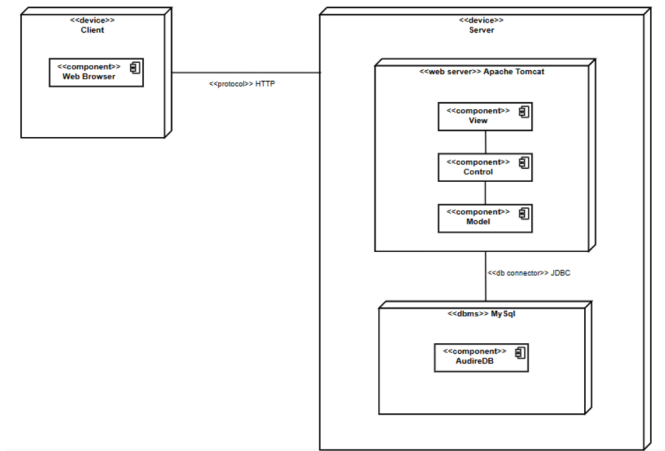


Figura: Schema di Deployment Client-Server

La gestione dei dati persistenti è affidata al DBMS relazionale **MySQL**, scelto per garantire affidabilità e integrità referenziale.

- **Struttura:** I dati sono organizzati in tabelle relazionali che mappano le entità principali
- **Proprietà ACID:** Le transazioni critiche rispettano i criteri di Atomicità, Consistenza, Isolamento e Durabilità
- **Sicurezza del Dato:** Le password sono memorizzate esclusivamente sotto forma di hash (Argon2id)

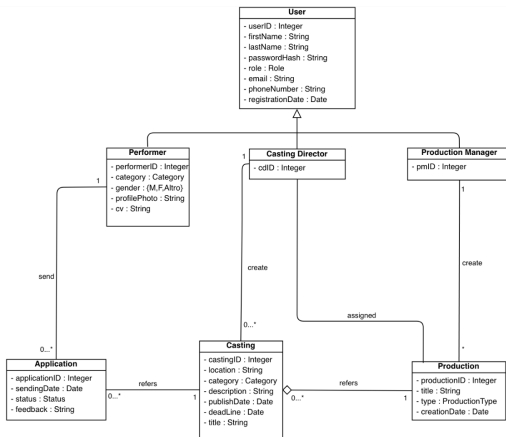


Figura: Diagramma Entità-Relazione

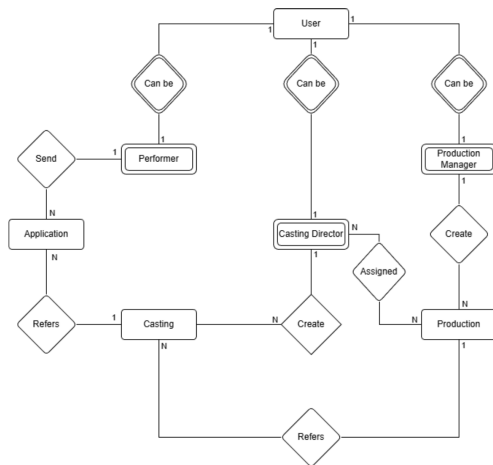


Figura: Schema Relazionale del Database

Attributo	Tipo	Vincolo di chiave
applicationID	INT	Primary Key
sendingDate	DATE	
status	ENUM('In attesa', 'Shortlist', 'Selezionata', 'Rifiutata')	
feedback	TEXT	
performerID	INT	Foreign Key
castingID	INT	Foreign Key

Oggetto/Attore	Performer	Casting Director	Production Manager
User	registration(), login(), update-Profile()	registration(), login(), update-Profile()	registration(), login(), updateProfile()
Production	<i>Nessun accesso</i>	getProductions() (solo lettura)	addProduction(), updateProduction(), removeProduction()
Team	<i>Nessun accesso</i>	<i>Nessun accesso</i>	assignCD(), removeCD()
Casting	searchCastings()	addCasting(), updateCasting(), removeCasting()	overviewCasting()
Application	sendApplication(), viewApplications()	getApplications(), updateStatus()	overviewProduction()

Start-up

All'avvio, il server inizializza le connessioni al database MySQL e avvia i servizi web. Il sistema verifica l'integrità dei file di configurazione e la disponibilità delle risorse di rete.

Start-up (post-fallimento)

In caso di riavvio dopo un crash, il sistema esegue un controllo di consistenza del database (rollback di transazioni incomplete) e ripristina lo stato operativo dai log di sistema.

Terminazione

La chiusura controllata prevede la disconnessione ordinata degli utenti attivi, il completamento delle transazioni in corso e lo spegnimento sicuro dei servizi server e database.

Fallimento

In caso di errore critico (es. perdita connessione DB), il sistema tenta di salvare lo stato nei log di errore, notifica l'amministratore e forza un riavvio o uno spegnimento di sicurezza per prevenire corruzione dati.

L'Object Design è la fase finale prima dell'implementazione vera e propria.
Questo documento:

- Descrive la struttura interna delle classi e le loro interfacce

Output dell'Object Design

Codice pronto per essere implementato, con specifiche complete e non ambigue

L'Object Design è la fase finale prima dell'implementazione vera e propria. Questo documento:

- Descrive la struttura interna delle classi e le loro interfacce
- Definisce i contratti (pre/post-condizioni) dei metodi usando OCL

Output dell'Object Design

Codice pronto per essere implementato, con specifiche complete e non ambigue

L'Object Design è la fase finale prima dell'implementazione vera e propria. Questo documento:

- Descrive la struttura interna delle classi e le loro interfacce
- Definisce i contratti (pre/post-condizioni) dei metodi usando OCL
- Specifica i design pattern utilizzati

Output dell'Object Design

Codice pronto per essere implementato, con specifiche complete e non ambigue

L'Object Design è la fase finale prima dell'implementazione vera e propria. Questo documento:

- Descrive la struttura interna delle classi e le loro interfacce
- Definisce i contratti (pre/post-condizioni) dei metodi usando OCL
- Specifica i design pattern utilizzati
- Stabilisce la mappatura tra modello a oggetti e persistenza

Output dell'Object Design

Codice pronto per essere implementato, con specifiche complete e non ambigue

Durante l'object design sono stati considerati ulteriori trade-off rispetto al system design:

Trade-off	Scelta
Performance vs Sicurezza	Tempi di risposta leggermente superiori per garantire controlli rigorosi e protezione dati utenti
Comprensibilità vs Tempo	Codice più commentato e strutturato aumenta il tempo di sviluppo ma facilita testing e manutenzione
Information Hiding vs Efficienza	Incapsulamento completo rende il codice più sicuro accettando un rallentamento impercettibile

Il codice è organizzato seguendo una struttura gerarchica che separa nettamente:

- **Presentation Layer:** JSP, CSS, JavaScript (WebContent)
- **Application Layer:** Controller per ogni sottosistema funzionale
- **Data Layer:** Model contenente DAO e DTO
- **Utilities:** Classi condivise tra i moduli

Ogni sottosistema funzionale ha un proprio package dedicato nel controller, mantenendo alta la coesione e basso l'accoppiamento.

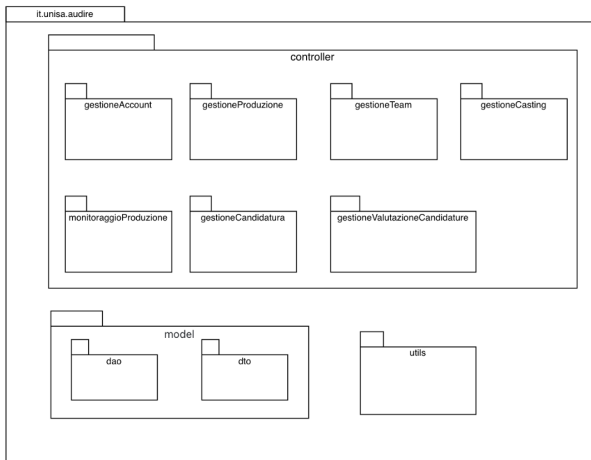


Figura: Organizzazione gerarchica dei package del sistema

Ogni classe DTO (JavaBeans per il trasporto dati tra i vari livelli del sistema) corrisponde a una tabella del database e garantisce serializzabilità:

- UserDTO
- PerformerDTO
- CastingDirectorDTO
- ProductionManagerDTO
- ProductionDTO
- CastingDTO
- ApplicationDTO
- TeamDTO

Caratteristiche comuni

Attributi privati, getter/setter pubblici, costruttore di default

Ogni classe DAO gestisce le operazioni CRUD (Create, Read, Update, Delete) per una specifica entità:

- **UserDAO**
- **PerformerDAO**
- **CastingDirectorDAO**
- **ProductionManagerDAO**
- **ProductionDAO**
- **CastingDAO**
- **ApplicationDAO**
- **TeamDAO**

Tutti i DAO stabiliscono connessione tramite DataSource e mappano ResultSet in DTO.

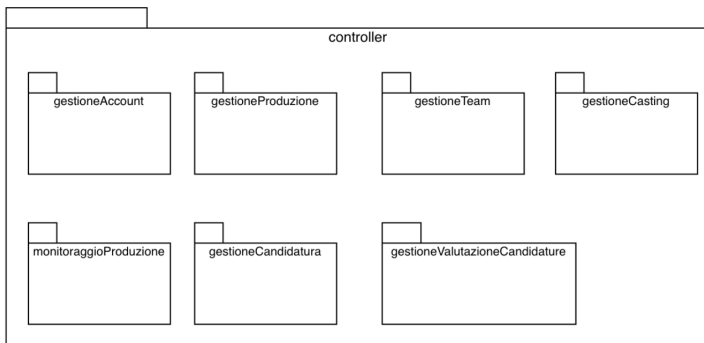


Figura: Decomposizione del package controller

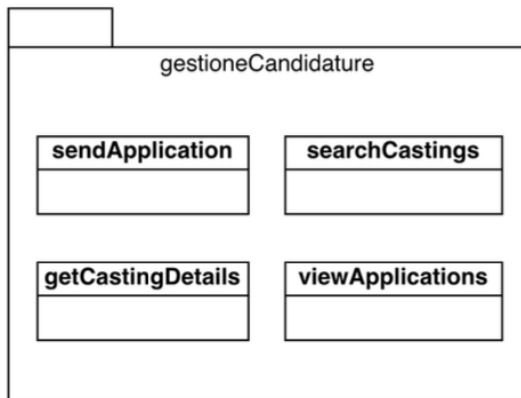


Figura: Classi del sottosistema Gestione Candidature

Ogni DAO espone un'interfaccia ben definita con metodi standard:

Operazioni CRUD Base

- `save(DTO object) : void` - Inserimento o aggiornamento
- `delete(Integer id) : boolean` - Eliminazione
- `getByID(Integer id) : DTO` - Recupero per chiave primaria
- `getAll(String order) : Collection<DTO>` - Lista completa

Metodi specializzati in base all'entità (es. `getByEmail` per `User`, `hasApplied` per `Application`).

Metodi Principali

- `save(UserDTO user) : void`
- `delete(Integer userID) : boolean`
- `getByID(Integer userID) : UserDTO`
- `getByEmail(String email) : UserDTO`
- `getAll(String order) : Collection<UserDTO>`

Metodi di Sicurezza

- `hashPassword(String plainPassword) : String`
- `verifyPassword(String plainPassword, String storedHash) : boolean`

Utilizza Argon2 per l'hashing sicuro delle password

Metodi Standard

- `save(ApplicationDTO app) : void`
- `delete(Integer applicationID) : boolean`
- `getByID(Integer applicationID) : ApplicationDTO`
- `getAll(String order) : Collection<ApplicationDTO>`

Metodi Specializzati

- `getByPerformerID(Integer performerID) : Collection<ApplicationDTO>`
- `getByCastingID(Integer castingID) : Collection<ApplicationDTO>`
- `hasApplied(Integer performerID, Integer castingID) : boolean`

OCIL è utilizzato per specificare contratti in modo formale e non ambiguo:

- **Pre-condizioni:** Vincoli che devono essere veri prima dell'esecuzione
- **Post-condizioni:** Garanzie sullo stato dopo l'esecuzione
- **Invarianti:** Proprietà che rimangono sempre vere

Vantaggi

- Elimina ambiguità nella specifica
- Facilita la verifica formale
- Documenta le aspettative in modo preciso

Descrizione

Rende persistente un oggetto Utente. Se l'ID è 0, esegue INSERT con nuovo ID. Se l'ID è positivo, esegue UPDATE.

Pre-condizione

`user  $\neq$  null`

Context

`UserDAO::save(user)`

Post-condizione

`(user.userID == 0  $\Rightarrow$  utenti $\rightarrow$ includes(user) con nuovo ID)`

$\vee$

`(user.userID > 0  $\Rightarrow$  utenti $\rightarrow$ select(u|u.id == user.userID).data == user.data)`

Descrizione

Verifica se esiste già una candidatura per la combinazione specifica di Performer e Casting. Previene duplicati.

Context

```
ApplicationDAO::hasApplied(performerID, castingID)
```

Pre-condizione

```
performerID > 0  $\wedge$  castingID > 0
```

Post-condizione

```
result == applications→exists(a | a.performerID == performerID  
 $\wedge$  a.castingID == castingID)
```

Ritorna true se la candidatura esiste, false altrimenti

Nel progetto Audire sono stati applicati tre design pattern fondamentali:

① MVC (Model-View-Controller)

Definizione di Design Pattern

Un design pattern descrive un problema che si verifica ripetutamente e propone il nucleo della soluzione, utilizzabile innumerevoli volte senza mai applicarla due volte allo stesso modo.

Nel progetto Audire sono stati applicati tre design pattern fondamentali:

- 1 **MVC (Model-View-Controller)**
- 2 **DAO (Data Access Object)**

Definizione di Design Pattern

Un design pattern descrive un problema che si verifica ripetutamente e propone il nucleo della soluzione, utilizzabile innumerevoli volte senza mai applicarla due volte allo stesso modo.

Nel progetto Audire sono stati applicati tre design pattern fondamentali:

- 1 **MVC (Model-View-Controller)**
- 2 **DAO (Data Access Object)**
- 3 **Singleton**

Definizione di Design Pattern

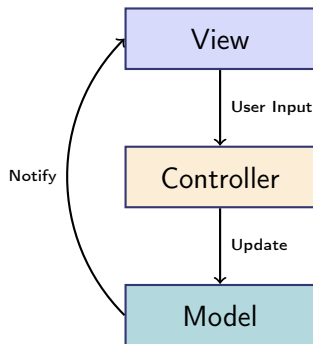
Un design pattern descrive un problema che si verifica ripetutamente e propone il nucleo della soluzione, utilizzabile innumerevoli volte senza mai applicarla due volte allo stesso modo.

Il pattern MVC separa un'applicazione in tre componenti distinti:

- **Model:** Gestione dati e logica di business
- **View:** Presentazione delle informazioni (JSP)
- **Controller:** Gestione input utente (Servlet)

Vantaggi:

- Separazione delle responsabilità
- Manutenibilità
- Testabilità



Scopo

Astrarre e incapsulare l'accesso alla fonte dati. Il DAO gestisce la connessione con la fonte dati per ottenere e memorizzare dati.

Vantaggi in Audire:

- Separazione completa tra logica di business e persistenza
- Facile sostituzione del DBMS senza modificare il controller
- Centralizzazione delle query SQL
- Mappatura automatica ResultSet $\leftrightarrow$ DTO

Problema

La creazione e chiusura di connessioni al database sono operazioni costose. Avere più istanze che gestiscono connessioni concorrentemente può portare a:

- Inconsistenze
- Esaurimento delle risorse
- Overhead prestazionale

Soluzione: Singleton su DataSource

Una sola istanza della classe DataSource gestisce il pool di connessioni, condivisa da tutti i DAO, garantendo:

- Accesso centralizzato
- Ottimizzazione risorse
- Thread-safety

Il testing rappresenta una fase critica del ciclo di vita del software, finalizzata a garantire la **qualità** e la **conformità** del sistema ai requisiti.

Documenti di Testing prodotti

- Test Plan
- Test Case Specification (TCS)
- Unit Test Document
- Integration Test Document
- Test Incident Report (TIR)
- Test Summary Report (TSR)

Lo scopo del Piano di Test è definire la strategia, le risorse e l'approccio per la verifica e la validazione del sistema software Audire.

Obiettivi del Test Plan

- Verificare che il sistema soddisfi i requisiti funzionali
- Identificare e correggere difetti
- Rispettare i vincoli di design

Il Test Plan definisce la strategia, le risorse e l'approccio per la verifica e la validazione del sistema Audire.

Approccio a Tre Livelli

- 1 **Unit Testing:** Verifica dei singoli componenti isolati (DAO, metodi)
- 2 **Integration Testing:** Test delle interazioni tra componenti integrate
- 3 **System Testing:** Verifica dei flussi completi basati sugli Use Case del RAD

L'obiettivo principale è garantire che i sottosistemi implementati soddisfino i requisiti funzionali descritti nel RAD e rispettino i vincoli di design definiti nell'ODD.

Verranno testate le funzionalità core dei sottosistemi implementati:

- 1 Gestione Account: Registrazione, Login.

Verranno testate le funzionalità core dei sottosistemi implementati:

- 1 Gestione Account: Registrazione, Login.
- 2 Gestione Produzioni: Creazione e Modifica di una produzione.

Verranno testate le funzionalità core dei sottosistemi implementati:

- 1 Gestione Account: Registrazione, Login.
- 2 Gestione Produzioni: Creazione e Modifica di una produzione.
- 3 Gestione Team: Assegnazione di un casting director a una produzione.

Verranno testate le funzionalità core dei sottosistemi implementati:

- 1 Gestione Account: Registrazione, Login.
- 2 Gestione Produzioni: Creazione e Modifica di una produzione.
- 3 Gestione Team: Assegnazione di un casting director a una produzione.
- 4 Gestione Casting: Creazione e Modifica dati di un annuncio.

Verranno testate le funzionalità core dei sottosistemi implementati:

- 1 Gestione Account: Registrazione, Login.
- 2 Gestione Produzioni: Creazione e Modifica di una produzione.
- 3 Gestione Team: Assegnazione di un casting director a una produzione.
- 4 Gestione Casting: Creazione e Modifica dati di un annuncio.
- 5 Gestione Candidature: Invio candidatura da parte del Performer.

Verranno testate le funzionalità core dei sottosistemi implementati:

- 1 Gestione Account: Registrazione, Login.
- 2 Gestione Produzioni: Creazione e Modifica di una produzione.
- 3 Gestione Team: Assegnazione di un casting director a una produzione.
- 4 Gestione Casting: Creazione e Modifica dati di un annuncio.
- 5 Gestione Candidature: Invio candidatura da parte del Performer.
- 6 Gestione Valutazione Candidature: Aggiornamento stato di una candidatura

Il metodo **Category Partition** è stato applicato per generare sistematicamente i casi di test funzionali.

Processo

- 1 Identificazione dei parametri di input (es. Email, Password, Ruolo)
- 2 Definizione delle categorie per ogni parametro
- 3 Identificazione delle scelte per ogni categoria
- 4 Definizione di vincoli e proprietà
- 5 Generazione dei test frame combinando le scelte

Questo approccio garantisce una copertura sistematica delle combinazioni di input rilevanti, riducendo il numero di test necessari mantenendo alta l'efficacia.

Param.	Cat.	Scelte	Cod.	Proprietà
Email	Fmt	Formato Errato	L_E1	[error]
		Formato Valido	L_E2	[ValidFmt]
	DB	Non trovata	L_D1	[error][if ValidFmt]
		Trovata	L_D2	[UserFound]
Psw	Match	Errata (non coincide)	L_P1	[error]
		Corretta	L_P2	[ValidPass]
Ruolo	Tipo	Casting Director.	L_R1	[isCd]
		Performer	L_R2	[isPerf]
		Production Manager	L_R3	[isPm]

ID Test	Scenario	Test Frame	Esito
TC_Auth_1	Email malformata	L_E1, L_P2, L_R1	Errato (Msg fmt)
TC_Auth_2	Utente non esistente	L_E2, L_D1, L_P2, L_R1	Errato (User not found)
TC_Auth_3	Password errata	L_E2, L_D2, L_P1, L_R1	Errato (Wrong Psw)
TC_Auth_4	Login Cd OK	L_E2, L_D2, L_P2, L_R1	Corretto (Dash. Cd)
TC_Auth_5	Login Pm OK	L_E2, L_D2, L_P2, L_R3	Corretto (Dash. Pm)
TC_Auth_6	Login Perf. OK	L_E2, L_D2, L_P2, L_R2	Corretto (Dash. Perf)

Ogni test case combina una scelta da ogni categoria per verificare comportamenti specifici del sistema.

Approccio Black-Box

Verifica del comportamento I/O senza esaminare la struttura interna delle componenti

Framework: JUnit

Database in-memory H2 per isolare i test dall'ambiente di produzione

Classi DAO Testate:

- UserDAO, PerformerDAO, CastingDirectorDAO
- ProductionManagerDAO, ProductionDAO
- CastingDAO, TeamDAO, ApplicationDAO



The screenshot shows the output of a JUnit test run for the UserDAOTest class. On the left, a list of test methods is shown with green checkmarks indicating they all passed. On the right, a summary line states '6 tests passed' and '6 tests total, 25 ms'. Below this, the file path to the Java executable is shown, and a message indicates the process finished with exit code 0.

Test Method	Duration
testSave()	15 ms
testGetById()	4 ms
testGetByEmail()	1 ms
testGetAll()	2 ms
testUpdate()	2 ms
testDelete()	1 ms

✓ 6 tests passed 6 tests total, 25 ms

Process finished with exit code 0

Figura: Esempio di test case JUnit per UserDAO

Strategia Big-Bang

Tutti i moduli sviluppati (DAO, Bean, Servlet, Pagine JSP) vengono integrati nel sistema contemporaneamente e il sistema viene testato nella sua interezza.

Vantaggi:

- Velocità di esecuzione
- Realismo: test in ambiente simile alla produzione

Svantaggi:

- Difficoltà nell'isolamento degli errori
- Rischio di blocco critico se un modulo fallisce

Tool utilizzato: Selenium IDE per l'automazione dei test funzionali

- **TC 1.1.1 - TC 1.1.8:** Registrazione con diverse combinazioni di input
- **TC Auth 1 - TC Auth 6:** Login con credenziali valide/invalidi per ogni ruolo
- **TC Prod 1 - TC Prod 3 & TC Edit 1 - TC Edit 4:** Creazione e modifica produzioni
- **TC Team 1 - TC Team 6:** Gestione team di produzione
- **TC Cast 1 - TC Cast 7 & TC M Cast 1 - TC M Cast 4:** Pubblicazione annunci casting e modifica
- **TC App 1 - TC App 3:** Invio candidature
- **TC Val 1 - TC Val 4:** Valutazione candidature

L'obiettivo di questa fase è dimostrare che il sistema rispetta i requisiti individuati durante la fase di analisi.

Test Case ID	TC_Auth_1
Test Frame	L_E1, L_P2, L_R1
Pre-condizione	L'utente si trova nella pagina di login e non è autenticato.
Flusso di eventi	1. L'utente inserisce nel campo email la stringa "utente.senza.chiocciola" (o altro formato invalido) e digita una password sintatticamente valida nel campo password. 2. L'utente clicca sul bottone "Accedi".
Oracolo	Il login non viene eseguito. Il sistema evidenzia il campo "Email" con un contorno di colore rosso per indicare il formato invalido.

Test Case ID	TC_App_3
Test Frame	DL_OK, H_NEW
Pre-condizione	Il Performer è loggato, il casting è aperto (non scaduto) e l'utente non si è ancora candidato.
Flusso di eventi	1. Il Performer visualizza i dettagli del casting e clicca sul bottone "Candidati Ora". 2. Nel modale di conferma, verifica che i propri dati (Foto profilo e CV) siano corretti e conferma l'invio.
Oracolo	La candidatura viene inviata con successo. Il sistema crea un record nella tabella Application con stato iniziale " In Attesa ". L'utente riceve una conferma visiva.

Copertura: 100% dei test pianificati eseguiti

4 Test Case con esito **PASSED** (comportamento diverso dall'atteso):

- 1 **TC Team 1:** Messaggio di errore impreciso (ordine dei controlli errato)
- 2 **TC Cast 5:** Vulnerabilità Broken Access Control - creazione casting non autorizzata
- 3 **TC M Cast 2:** CD rimosso dal team poteva ancora modificare casting
- 4 **TC Val 2:** Violazione logica di stato - candidatura "Rifiutata" riportabile a "In Attesa"

✓ **Tutti i bug sono stati corretti e documentati nel Test Incident Report**

Test Case	Problema	Soluzione
TC Team 1	Messaggio "Accesso negato" invece di "Produzione non trovata"	Riordinati i controlli nella Servlet: prima verificare esistenza risorsa, poi permessi
TC Cast 5	Creazione casting da utenti non autorizzati forzando URL	Aggiunto controllo di appartenenza al team prima di ogni operazione CRUD
TC M Cast 2	CD rimosso poteva modificare casting creati in precedenza	Implementato controllo real-time dell'appartenenza al team ad ogni richiesta
TC Val 2	Candidatura "Rifiutata" riportabile a "In Attesa"	Implementato state machine che impedisce transizioni illegali

Questi fallimenti hanno permesso di identificare vulnerabilità di sicurezza e logiche errate nel backend che non erano emerse durante Unit e Integration Test.

L'attività di System Testing su Audire ha portato alla luce difetti critici riguardanti principalmente la **sicurezza** e la **gestione dei permessi**.

Grazie alle correzioni apportate a seguito dei fallimenti riportati, il sistema risulta ora **stabile** e **conforme** ai requisiti specificati nel RAD.

Grazie per l'attenzione

? Domande?

Gennaro Carmine Tozza - 0512120382

Gabriele Malanga - 0512119344